

FRD

Functional Requirements Document (FRD)

AWS Glue Data Pipeline – Customer Order Processing

•

1. Document Overview

Purpose

This Functional Requirements Document defines the functional behavior of an AWS Glue-based data pipeline that ingests customer and order data from S3, performs data quality operations, implements SCD Type 2 logic using Apache Hudi, and produces analytical aggregates for querying via Amazon Athena.

Scope

This FRD covers:

- Ingestion of customer and order CSV files from S3
- Data cleaning and deduplication
- Registration of datasets in AWS Glue Data Catalog
- Implementation of SCD Type 2 for order summary using Hudi format
- Incremental processing triggered by customer updates
- Generation of customer aggregate spend analytics table
- Storage of curated and analytics datasets in S3

This FRD does NOT cover:

- Detailed AWS IAM policies or security configurations
- Specific CloudWatch alarm thresholds
- Lake Formation governance rules
- Downstream BI tool integrations

Assumptions

- CSV files in source S3 paths follow the schemas defined in Section 2
- Source files are UTF-8 encoded with comma delimiters
- "Null" (string) and NULL (empty) values are considered invalid and must be removed
- Customer updates are identified by changes in any customer attribute (Name, EmailId, Region)
- Order data contains valid CustId references to existing customers
- The AWS Glue Data Catalog database `gen\_ai\_poc\_databrickscoe` already exists
- S3 bucket `adif-sdlc` and required paths are provisioned with appropriate permissions

- Date fields in Order data are in a parseable format (assumed ISO 8601 or similar)
- PricePerUnit and Qty are numeric values
- TotalAmount in aggregation is calculated as $\text{PricePerUnit} \times \text{Qty}$
-

2. Functional Requirements

FR-INGEST-001: Ingest Customer Data from S3

- Title: Load Customer CSV Files from S3
- Description:

The system shall read customer data from the specified S3 path and load it into a structured format for processing.

- Business Objective:

Enable downstream processing and analytics by making customer master data available in the pipeline.

- Inputs:
- Source: `s3://adif-sdlc/sdlc\_wizard/customerdata/`
- Format: CSV
- Schema:

	Column Name	Data Type	Description	
	-----	-----	-----	
	CustId	String	Unique customer identifier	
	Name	String	Customer full name	
	EmailId	String	Customer email address	
	Region	String	Customer geographic region	

- Processing Rules / Functional Steps:
 1. Read all CSV files from `s3://adif-sdlc/sdlc\_wizard/customerdata/`
 2. Parse CSV with header row
 3. Load data into a Spark DataFrame with the defined schema
- Outputs:
- In-memory customer dataset ready for cleaning
- Preconditions:
 - S3 path exists and contains at least one CSV file
 - AWS Glue job has read permissions on the S3 path

- Postconditions:
- Customer data is loaded into memory for further processing
- Error / Validation Conditions:
- If S3 path is empty or inaccessible, job must fail with clear error message
- If CSV structure does not match expected schema, job must fail
-

FR-INGEST-002: Ingest Order Data from S3

- Title: Load Order CSV Files from S3
- Description:

The system shall read order transaction data from the specified S3 path and load it into a structured format for processing.

- Business Objective:

Capture order transactions for join operations, SCD Type 2 tracking, and aggregation.

- Inputs:
- Source: `s3://adif-sdlc/sdlc\_wizard/orderdata/`
- Format: CSV
- Schema:

	Column Name	Data Type	Description	
	-----	-----	-----	
	OrderId	String	Unique order identifier	
	ItemName	String	Name of ordered item	
	PricePerUnit	Numeric	Price per unit of item	
	Qty	Numeric	Quantity ordered	
	Date	String	Order date	
	CustId	String	Foreign key to Customer	

- Processing Rules / Functional Steps:
 1. Read all CSV files from `s3://adif-sdlc/sdlc\_wizard/orderdata/`
 2. Parse CSV with header row
 3. Load data into a Spark DataFrame with the defined schema
- Outputs:
- In-memory order dataset ready for cleaning

- Preconditions:
- S3 path exists and contains at least one CSV file
- AWS Glue job has read permissions on the S3 path
- Postconditions:
- Order data is loaded into memory for further processing
- Error / Validation Conditions:
- If S3 path is empty or inaccessible, job must fail with clear error message
- If CSV structure does not match expected schema, job must fail
-

FR-CLEAN-001: Remove Null and Duplicate Values

- Title: Data Quality Cleaning for Customer and Order Datasets
- Description:

The system shall remove rows containing "Null" (string literal) or NULL (empty) values, and eliminate duplicate records from both customer and order datasets.

- Business Objective:

Ensure data quality and consistency before downstream processing and storage.

- Inputs:
- In-memory customer dataset (from FR-INGEST-001)
- In-memory order dataset (from FR-INGEST-002)
- Processing Rules / Functional Steps:
- 1. For customer dataset:
 - Remove rows where any column contains the string "Null" (case-insensitive) or NULL/empty value
 - Remove duplicate rows based on all columns
- 2. For order dataset:
 - Remove rows where any column contains the string "Null" (case-insensitive) or NULL/empty value
 - Remove duplicate rows based on all columns
- Outputs:
- Cleaned customer dataset
- Cleaned order dataset

- Preconditions:
- Customer and order datasets are loaded in memory
- Postconditions:
- Both datasets contain only valid, non-null, non-duplicate records
- Error / Validation Conditions:
- If all records are removed due to null values, log warning but continue processing
- If no duplicates found, continue without error
-

FR-CATALOG-001: Register Cleaned Datasets in Glue Data Catalog

- Title: Create Glue Catalog Tables for Customer and Order
- Description:

The system shall register the cleaned customer and order datasets as tables in the AWS Glue Data Catalog for query access via Athena.

- Business Objective:

Enable SQL-based querying and metadata management of source datasets.

- Inputs:
- Cleaned customer dataset (from FR-CLEAN-001)
- Cleaned order dataset (from FR-CLEAN-001)
- Processing Rules / Functional Steps:
- 1. Write cleaned customer data to S3 in Parquet format
- 2. Register as Glue Catalog table:
 - Database: `gen\_ai\_poc\_databrickscoe`
 - Table: `sdhc\_wizard\_customer`
- 3. Write cleaned order data to S3 in Parquet format
- 4. Register as Glue Catalog table:
 - Database: `gen\_ai\_poc\_databrickscoe`
 - Table: `sdhc\_wizard\_order`
- Outputs:
- Glue Catalog table: `gen\_ai\_poc\_databrickscoe.sdhc\_wizard\_customer`
- Glue Catalog table: `gen\_ai\_poc\_databrickscoe.sdhc\_wizard\_order`
- Underlying S3 data in Parquet format

- Preconditions:
- Database `gen\_ai\_poc\_databrickscoe` exists in Glue Data Catalog
- AWS Glue job has permissions to create/update catalog tables
- Postconditions:
- Tables are queryable via Athena
- Metadata is registered in Glue Data Catalog
- Error / Validation Conditions:
- If database does not exist, job must fail
- If table creation fails, job must fail with error details
-

FR-SCD2-001: Implement SCD Type 2 for Order Summary

- Title: Create and Maintain Order Summary with SCD Type 2 Logic Using Hudi
- Description:

The system shall join customer and order data, apply SCD Type 2 logic to track historical changes, and store the result in Apache Hudi format on S3. When customer attributes change, new active rows are inserted and previous rows are closed with end dates.

- Business Objective:

Maintain complete historical tracking of order summaries with customer dimension changes over time.

- Inputs:
- Cleaned customer dataset (from FR-CLEAN-001)
- Cleaned order dataset (from FR-CLEAN-001)
- Existing `ordersummary` Hudi table (for incremental updates)
- Processing Rules / Functional Steps:
- 1. Join customer and order datasets on `CustId`
- 2. For each joined record:
 - If customer attributes (Name, EmailId, Region) have changed since last load:
 - Insert new row with `IsActive = True`, `StartDate = current\_timestamp`, `EndDate = NULL`, `OpTs = current\_timestamp`
 - Update previous row: set `IsActive = False`, `EndDate = current\_timestamp`
 - If no change detected:
 - Keep existing active row unchanged
- 3. Write result to Hudi table with upsert operation

4. Register/update Glue Catalog table:

- Database: `gen\_ai\_poc\_databrickscoe`
- Table: `ordersummary`
- Location: `s3://adif-sdlc/curated/sdlc\_wizard/ordersummary/`
- Outputs:
 - Hudi table: `ordersummary` at `s3://adif-sdlc/curated/sdlc\_wizard/ordersummary/`
 - Glue Catalog table: `gen\_ai\_poc\_databrickscoe.ordersummary`
 - Schema includes: OrderId, ItemName, PricePerUnit, Qty, Date, CustId, Name, EmailId, Region, IsActive, StartDate, EndDate, OpTs
- Preconditions:
 - Customer and order datasets are cleaned and joined successfully
 - S3 path `s3://adif-sdlc/curated/sdlc\_wizard/ordersummary/` is accessible
 - Hudi libraries are available in Glue job environment
- Postconditions:
 - All current and historical order summaries are stored with SCD Type 2 attributes
 - Only one active row per unique order-customer combination exists
 - Historical rows have `IsActive = False` and valid `EndDate`
- Error / Validation Conditions:
 - If join produces no records, log warning and skip Hudi write
 - If Hudi write fails, job must fail with error details
 - If customer has no matching orders, no ordersummary record is created
-

FR-INCR-001: Incremental Processing Based on Customer Updates

- Title: Trigger SCD Type 2 Updates When Customer Data Changes
- Description:

The system shall detect changes in customer attributes and trigger incremental updates to the `ordersummary` table, maintaining SCD Type 2 history.

- Business Objective:

Ensure order summary reflects the most current customer information while preserving historical accuracy.

- Inputs:
 - New/updated customer data (from FR-INGEST-001 and FR-CLEAN-001)

- Existing `ordersummary` Hudi table
- Processing Rules / Functional Steps:
 1. Compare incoming customer records with existing customer records in catalog
 2. Identify changed records (CustId match with different Name, EmailId, or Region)
 3. For each changed customer:
 - Retrieve all related orders from `sdlc\_wizard\_order`
 - Apply SCD Type 2 logic as defined in FR-SCD2-001
 4. Perform Hudi upsert to update `ordersummary`
- Outputs:
 - Updated `ordersummary` Hudi table with new active rows and closed historical rows
- Preconditions:
 - Customer data has been ingested and cleaned
 - `ordersummary` table exists
- Postconditions:
 - All affected order summary records reflect customer changes
 - SCD Type 2 attributes (IsActive, StartDate, EndDate, OpTs) are correctly maintained
- Error / Validation Conditions:
 - If no customer changes detected, skip SCD Type 2 processing
 - If Hudi upsert fails, job must fail
-

FR-AGG-001: Generate Customer Aggregate Spend Table

- Title: Create Customer Aggregate Spend Analytics Table
- Description:

The system shall aggregate order data by customer to calculate total spend and store the result in a dedicated analytics table.

- Business Objective:

Provide a pre-aggregated view of customer spending for fast analytical queries.

- Inputs:
 - Cleaned customer dataset (from FR-CLEAN-001)
 - Cleaned order dataset (from FR-CLEAN-001)

- Processing Rules / Functional Steps:

1. Join customer and order datasets on `CustId`
2. Calculate `TotalAmount = PricePerUnit × Qty` for each order
3. Group by customer `Name` and order `Date`
4. Sum `TotalAmount` for each group
5. Write result to S3 in Parquet format at `s3://adif-sdlc/analytics/customeraggregatespend/`
6. Register/update Glue Catalog table:

- Database: `gen\_ai\_poc\_databrickscoe`

- Table: `customeraggregatespend`

- Outputs:

- Glue Catalog table: `gen\_ai\_poc\_databrickscoe.customeraggregatespend`

- Location: `s3://adif-sdlc/analytics/customeraggregatespend/`

- Schema:

	Column Name	Data Type	Description	
	-----	-----	-----	
	Name	String	Customer name	
	TotalAmount	Numeric	Sum of (PricePerUnit × Qty)	
	Date	String	Order date	

- Preconditions:

- Customer and order datasets are cleaned and available
- S3 path `s3://adif-sdlc/analytics/customeraggregatespend/` is accessible

- Postconditions:

- Aggregate table is queryable via Athena
- Data reflects total spend per customer per date

- Error / Validation Conditions:

- If join produces no records, create empty table with schema
- If aggregation fails, job must fail with error details

-

3. Non-Functional Considerations

Performance

- AWS Glue jobs should leverage Spark partitioning and parallelism for large datasets
- Hudi upsert operations should use appropriate record keys and precombine keys for efficiency

Monitoring

- AWS CloudWatch should be used to monitor Glue job execution, duration, and failures
- Logs should capture data quality metrics (e.g., number of nulls removed, duplicates dropped)

Scalability

- Pipeline should handle growing data volumes without manual intervention
- S3 storage and Glue DPU allocation should scale with data size

Data Governance (Optional)

- AWS Lake Formation may be used to manage access control and data cataloging
- Metadata lineage should be traceable through Glue Data Catalog
-

4. Requirement Traceability Notes

FRD Requirement	BRD Section	Description	
-----	-----	-----	
FR-INGEST-001	Section 1, 2	Customer CSV ingestion from `s3://adif-sdlc/sdlc_wizard/customerdata/`	
FR-INGEST-002	Section 1, 2	Order CSV ingestion from `s3://adif-sdlc/sdlc_wizard/orderdata/`	
FR-CLEAN-001	Section 3	Data cleaning: remove "Null", NULL, and duplicates	
FR-CATALOG-001	Section 4	Register `sdlc_wizard_customer` and `sdlc_wizard_order` in Glue Catalog	
FR-SCD2-001	Section 5	SCD Type 2 implementation for `ordersummary` using Hudi	
FR-INCR-001	Section 6	Incremental logic triggered by customer updates	
FR-AGG-001	Section 7	Aggregation table `customeraggregatespend`	

All functional requirements are directly traceable to explicit statements in the BRD. No business logic has been invented or inferred beyond what is documented.

-
- End of Functional Requirements Document